

MESH CHAT

BY: HARSH SHARMA
ADVISOR: PROF. SHIVAKUMAR MATHAPATHI



Introduction

MeshChat is a real-time chat system that works without internet, Wi-Fi, or servers. Messages travel over Bluetooth Low Energy (BLE), hopping across a self-organizing mesh of smartphones and Raspberry Pi relay nodes.

It mainly has two components:



Problem Statement

Modern messaging is entirely dependent on centralized infrastructure. When it fails, communication fails.

- Cellular towers and internet are the first to go in emergencies
- Direct phone-to-phone Bluetooth range (30m) is too short for real deployment
- Smartphones cannot reliably advertise and scan BLE simultaneously

Since there is no lightweight, open, offline-first solution that addresses all of these together, Mesh Chat tries to answer them and provide an alternative that can overcome these shortcomings.

Limitation

Current implementation is best treated as a prototype. The mobile application needs clearer handling of BLE roles.

Methodology

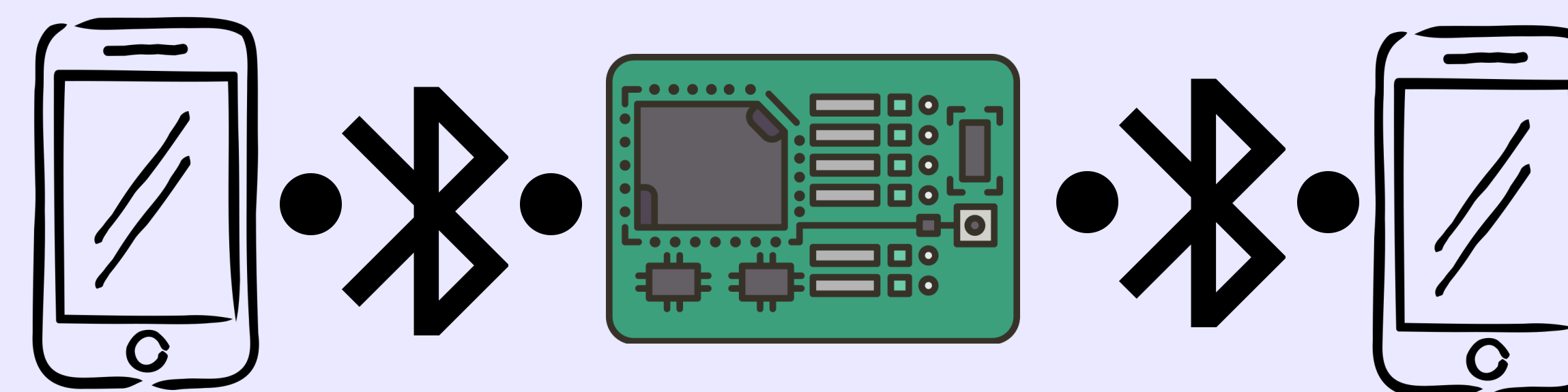
Devices discover each other via a shared BLE Service UUID. Raspberry Pi nodes simultaneously bridge all connected devices.

GATT Protocol

Each node hosts a GATT server with two characteristics:

- One for message delivery (write + notify),
- One for the device's display name (read).

Messages are compact JSON objects and are serialized to raw bytes within BLE's 512-byte Maximum Transmission Unit (MTU).



Mesh Relay Algorithm

- Every message carries a unique ID and a Time-to-Live (TTL) counter (default: 7)
- On receipt, each node checks a `seen\_ids` set so the duplicates are dropped
 - If $TTL > 1$, the message is forwarded to all other peers with $TTL - 1$
 - At $TTL = 0$, the message is discarded to prevent any infinite loops.

Why Raspberry Pi?

Not all devices can advertise BLE while backgrounded. Pi nodes run Linux's BlueZ stack, supporting simultaneous peripheral with central roles 24/7, acting as a permanent mesh backbone that phones cannot provide.

Tech Stack:

Flutter mobile app and Python-based Raspberry Pi relay nodes using BLE, BlueZ, flutter_blue_plus, bleak, etc.

Next Steps

A few improvements that can be made on top

- Improving the device's permission handling
- End-to-end encryption, shared room passphrase
- Improving reconnection, error recovery, and message history syncing
- Keeping BLE scanning alive when the app is backgrounded
- Adding delivery status and peer health indicators

Applications



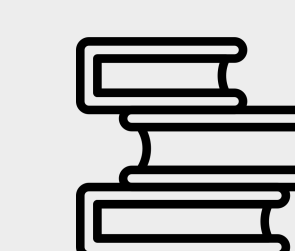
Emergency Response



Large Venue Events



Local/Private Community



Education



Limited Internet

Conclusion

This demonstrates how local, infrastructure-free communication can be built using BLE, Flutter, and Raspberry Pi. While the prototype faces practical BLE and mobile OS limitations, the project provides a strong foundation for offline communication. With improved reliability, security, and relay-node coordination, MeshChat could become a practical tool for communication in low-connectivity environments.